

Project Report

Performance Analysis of Sequential and Parallel Matrix Correlation Programs

This report presents the design, implementation, and performance evaluation of C++ programs developed to compute matrix correlation using sequential and parallel approaches. The performance comparison is carried out using Linux performance counters to understand CPU utilization, execution time, and efficiency.

1. Introduction

With the growing demand for high-performance computing, understanding the difference between sequential and parallel program execution has become essential. This project focuses on analyzing matrix correlation computation using both approaches and studying their runtime behavior using performance metrics.

2. Objectives

- To implement matrix correlation using a sequential approach.
- To implement a parallel version of the same algorithm.
- To compare execution time and CPU utilization.
- To analyze performance counters for better architectural understanding.

3. Methodology

The programs were implemented in C++ and executed on a Linux-based system. Matrix sizes of 1000×1000 were used for evaluation. The Linux *perf* tool was employed to collect detailed performance statistics such as task clock, CPU cycles, instructions per cycle, branch misses, and execution time.

4. Program Description

Sequential Implementation: Executes matrix correlation in a single-threaded manner, processing one instruction sequence at a time.

Parallel Implementation: Utilizes multiple CPU cores to divide the computation workload, leading to better resource utilization and reduced execution time for large datasets.

5. Results and Performance Analysis

The following figures show performance counter statistics captured during program execution, highlighting the difference between sequential and parallel implementations.

Figure 1: Performance statistics for sequential matrix correlation

```

Performance counter stats for './correlate_matrix_sequential 1000 1000':

    249,719,980    task-clock                #    1.100 CPUs utilized
         76       context-switches          #   304.341 /sec
         42       cpu-migrations            #   168.188 /sec
        6,132     page-faults              #   24.556 K/sec
    1,009,971,077  cpu_atom/instructions/                #    3.17  insn per cycle          (2.44%)
    2,087,233,577  cpu_core/instructions/                #    1.95  insn per cycle          (96.28%)
    318,505,070    cpu_atom/cycles/                    #    1.275 GHz                    (3.72%)
    1,072,855,627  cpu_core/cycles/                    #    4.296 GHz                    (96.28%)
    106,307,837    cpu_atom/branches/                  #   425.708 M/sec                 (3.72%)
    148,363,230    cpu_core/branches/                  #   594.118 M/sec                 (96.28%)
        300,528    cpu_atom/branch-misses/                #    0.28% of all branches        (3.72%)
        567,101    cpu_core/branch-misses/                #    0.38% of all branches        (96.28%)
#    64.4 % tma_backend_bound
#    0.8 % tma_bad_speculation
#    1.6 % tma_frontend_bound
#   33.2 % tma_retiring          (96.28%)
#   12.1 % tma_bad_speculation
#   68.2 % tma_retiring          (3.72%)
#    0.0 % tma_backend_bound
#   19.7 % tma_frontend_bound    (3.72%)

    0.226957115 seconds time elapsed

    0.230111000 seconds user
    0.023213000 seconds sys

```

Figure 2: Performance statistics for parallel matrix correlation

```

Performance counter stats for './correlate_matrix_parallel 1000 1000':

    12,415,210,763 task-clock                #    7.647 CPUs utilized
         2,087     context-switches          #   168.100 /sec
         475       cpu-migrations            #   38.260 /sec
        52,669     page-faults              #    4.242 K/sec
    67,639,508,186 cpu_atom/instructions/                #    1.79  insn per cycle          (30.82%)
   129,885,091,795 cpu_core/instructions/                #    2.64  insn per cycle          (64.73%)
    37,842,066,507 cpu_atom/cycles/                    #    3.048 GHz                    (30.88%)
    49,181,901,950 cpu_core/cycles/                    #    3.961 GHz                    (64.73%)
    2,020,643,931  cpu_atom/branches/                  #   162.756 M/sec                 (30.89%)
    3,965,426,557  cpu_core/branches/                  #   319.401 M/sec                 (64.73%)
         8,695,253 cpu_atom/branch-misses/                #    0.43% of all branches        (30.83%)
        15,872,929 cpu_core/branch-misses/                #    0.40% of all branches        (64.73%)
#   25.2 % tma_backend_bound
#    1.1 % tma_bad_speculation
#    7.0 % tma_frontend_bound
#   66.7 % tma_retiring          (64.73%)
#    1.7 % tma_bad_speculation
#   50.0 % tma_retiring          (30.85%)
#   47.0 % tma_backend_bound
#    1.3 % tma_frontend_bound    (30.88%)

    1.623454612 seconds time elapsed

    12.286495000 seconds user
     0.143203000 seconds sys

```

6. Comparative Analysis

From the performance data, it is observed that the sequential implementation utilizes approximately one CPU core, resulting in lower throughput but shorter elapsed time for a single run. In contrast, the parallel implementation effectively utilizes multiple CPU cores, leading to higher instruction throughput and improved scalability, although with higher total CPU time consumption.

7. Conclusion

This project successfully demonstrates the practical differences between sequential and parallel program execution. The performance analysis confirms that parallel computing significantly improves CPU utilization and computational efficiency for large matrix operations.